Group 2, Problem 11

Part A:

Q1

V2:

after	id array	# of entries overwritten
union(0,1)	id[1,1,2,3,4,5]	1
union(1,2)	id[2,2,2,3,4,5]	2
union(2,3)	id[3,3,3,3,4,5]	3
union(3,4)	id[4,4,4,4,4,5]	4
union(4,5)	id[5,5,5,5,5,5]	5

30 total entries checked

15 total entries overwritten

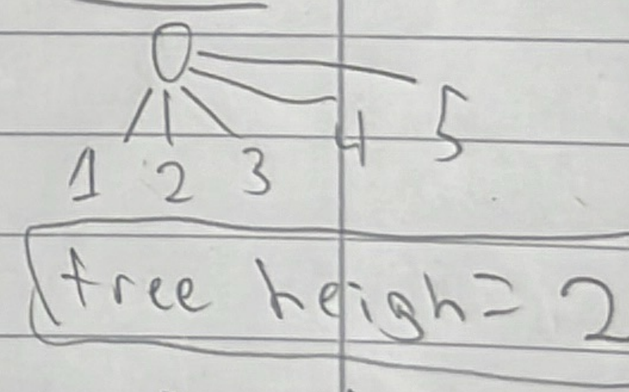
Bottleneck: The group being relabeled grows by 1 each time, so cost per union grows.
For n, 1+2+...+(n-1)
$= n(\frac{n-1}{2}) \rightarrow \boxed{\Theta(n^2)}$

V3:

after	parent array [0,1,2,3,4,5]
union(0,1)	id[1,1,2,3,4,5]
(1,2)	id[1,2,2,3,4,5]
(3,3)	id[1,2,3,3,4,5]
(3,4)	id[1,2,3,4,4,5]
(4,5)	id[1,2,3,4,5,5]

final parent[1,2,3,4,5,5]

tree



height = 6

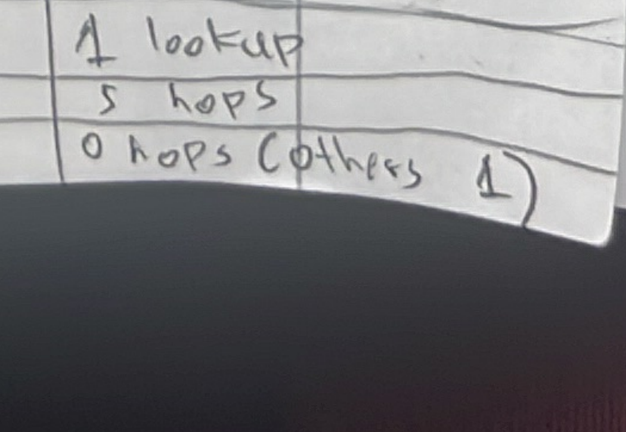
5 hops for find(0)

unions are cheap and cost 0 hops, but the bottleneck is nothing stops a tall tree from being hung under one node. We built worst tree so find is $\Theta(n)$

V4:

after	parent array [0,1,2,3,4,5]
union(0,1)	parent[0,0,2,3,4,5] ✓
rankstie	rank[1,0,0,0,0,0]
rank u>v (1,2)	parent[0,0,0,3,4,5]
	rank[2,0,0,0,0,0]
rank u>v (2,3)	parent[0,0,0,0,4,5]
	rank[3,0,0,0,0,0]
rank u>v (3,4)	parent[0,0,0,0,0,5]
	rank[4,0,0,0,0,0]
rank u>v (4,5)	parent[0,0,0,0,0,0]
	rank[5,0,0,0,0,0]

tree



tree height = 2

This fixes V3 bottleneck and makes it so a trees height only grows when 2 equal rank trees merge

Operation-count comparison

	Array writes during unions	hops/checks during unions	Final height	Find(0) cost
V2	15 overwrites	30 entries checked	n/a	1 lookup
V3	5	0 hops	5	5 hops
V4	5 parent + 1 rank	4 hops	1	0 hops (others 1)

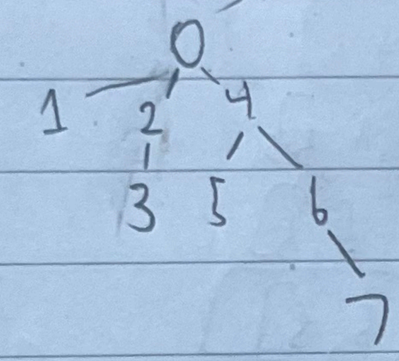
Group 2, Problem 11

Jonah Sandow

Q2:

step	ru (hops)	rv (hops)	rank compare	write	parent[] after	rank[] after
union(0,1)	0 (0)	1 (0)	0=0	parent[1]=0 rank[0]++	[0,0,2,3,4,5,6,7]	[1,0,0,0,0,0,0,0]
union(2,3)	2 (0)	3 (0)	0=0	parent[3]=2 rank[2]++	[0,0,2,2,4,5,6,7]	[1,0,1,0,0,0,0,0]
union(4,5)	4 (0)	5 (0)	0=0	parent[5]=4 rank[4]++	[0,0,2,2,4,4,6,7]	[1,0,1,0,1,0,0,0]
union(6,7)	6 (0)	7 (0)	0=0	parent[7]=6 rank[6]++	[0,0,2,2,4,4,6,6]	[1,0,1,0,1,0,1,0]
union(0,2)	0 (0)	2 (0)	1=1	parent[2]=0 rank[0]++	[0,0,0,2,4,4,6,6]	[2,0,1,0,1,0,1,0]
union(4,6)	4 (0)	6 (0)	1=1	parent[6]=4 rank[4]++	[0,0,0,2,4,4,4,6]	[2,0,1,0,2,0,1,0]
union(0,4)	0 (0)	4 (0)	2=2	parent[4]=0 rank[0]++	[0,0,0,2,0,4,4,6]	[3,0,1,0,2,0,1,0]

Resulting tree



after find_compressed(7): root = 0

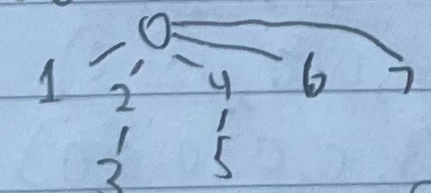
x=7, parent[7]=0

x=6, parent[6]=0

x=4, parent[4]=0

x=0, end loop and return

// new tree



tree height = 2

new parent array = [0,0,0,2,0,4,0,0]

Depths before and after find_compressed(7):

node	0	1	2	3	4	5	6	7
before	0	1	1	2	1	2	2	3
after	0	1	1	2	1	2	1	1

operations

union: 0 hops each

find(7): 3 hops

find_compression(7): 3 hops, 3 writes

after compression: find(7) is 1 hop

Bottleneck removed: v4 guarentees height $\leq \lceil \log_2 n \rceil$, but every find rewalks same path. v5 stores result of the walk directly in each node, so later finds on that path take 1 hop. Path compression pay small extra price once to flatten path, with makes later finds almost free.

Group 2, Problem 11, Q2 - Krustal MST trace

Q3: sort edges: (A, D, 5); (C, E, 5); (D, F, 6); (A, B, 7); (B, E, 7); (B, C, 8); (E, F, 8); (B, D, 9); (E, G, 9); (F, G, 11); (D, E, 15)

- using union by rank (V4): on tie v's root goes under u's root

#	Edge (w)	Find(u)	Find(v)	Accept/ reject	write	MST Edges so far
1	(A, D, 5)	find(A)=A	find(D)=D	Accept	parent[D]=A Rank[A]++	{A, D}, {B}, {C}, {E}, {F}, {G}
2	(C, E, 5)	find(C)=C	find(E)=E	Accept	Parent[E]=C Rank[C]++	{A, D}, {E, C}, {B}, {F}, {G}
3	(D, F, 6)	find(D)=A	find(F)=F	Accept	Parent[F]=A Rank[D]++	{A, D, F}, {E, C}, {B}, {G}
4	(A, B, 7)	Find(A)=A	find(B)=B	Accept	parent[B]=A Rank[A]++	{A, D, F, B}, {E, C}, {G}
5	(B, E, 7)	find(B)=A	find(E)=C	Accept	parent[C]=A	{A, D, F, B, E, C}, {G}
6	(B, C, 8)	find(B)=A	find(C)=A	reject		
7	(E, F, 8)	find(E)=A	find(F)=A	reject		
8	(B, D, 9)	find(B)=A	find(D)=A	reject		
9	(E, G, 9)	find(E)=A	find(G)=G	Accept	parent(G)=A	{A, D, F, B, E, G, G}
10	(F, G, 11)	find(F)=A	find(G)=A	reject		
11	(D, E, 15)	find(D)=A	find(E)=A	reject		

Final MST Edgest: (A, D, 5); (C, E, 5); (D, F, 6); (A, B, 7); (B, E, 7); (E, G, 9)

total weight = 39

Krustal's builds the MST greedily, taking the cheapest edges and only accepts the edge if its endpoints are in different components. union-find makes the cycle check 1 comparison: Find(u)==find(v). Operation count: 11 edges checked, 22 find calls, 6 unions, 5 rejections. Bottleneck eliminated: without union-find, each cycle needs a BFS/DFS over the tree built so far. That is O(V) per edge or O(E·V). with union by rank + path compression, each check is nearly O(1). Total cost is dominated by sorting edges O(E·logE)

Q4 Percolation and the Backwash Bug

1)

o (0,0)	x (0,1)	x (0,2)
o (1,0)	o (1,1)	x (1,2)
x (2,0)	o (2,1)	o (2,2)

- o = open cell
- x = Blocked cell

The percolating path is (0,0) → (1,0) → (1,1) → (2,1)

Converting the grid cells into indices

Formula:- index (r, c) = r × L + c

Here L = 3

Cell	Index
(0,0)	0
(1,0)	3
(1,1)	4
(2,1)	7
(2,2)	8
vtop	9
vbottom	10

Performing Union Operations

Operation	Purpose
Union (0,3)	Connect (0,0) and (1,0)
Union (3,4)	Connect (1,0) and (1,1)
Union (4,7)	Connect (1,1) and (2,1)

Connecting the virtual top node: Union (9,0)

Connecting the Vbottom node to open bottom-row cells

Union (10,7) Union (10,8)

Tracing the connected set
Connected component is {9, 0, 3, 4, 7, 10, 8}
Using cell coordinates
{vtop; (0,0), (1,0), (1,1), (2,1), vbottom, (2,2)}
∴ connected (vtop, (2,2)) = True

This can incorrectly label an isolated bottom row cell as full, even when no physical water path reaches it.

There is a mistake in the specific grid example (2,2) is open and directly adjacent to (2,1), which is also open. So in the grid, water actually can reach (2,2). It is not a completely isolated cell. The Union-find of backwash is still correct, but the example does not demonstrate it properly.

Fix - Use two Union-Find structures.

Structure	Virtual nodes	Purpose
uf-perc	vtop & vbottom	Check percolation
uf-full	vtop only	Check whether a cell is full

For uf-perc:
- Connect open top-row cells to vtop
- Connect open bottom-row cells to vbottom
- Connect neighbouring open cells
- Check Connected (vtop, vbottom)

For uf-full:
- Connect open top-row cells to vtop
- Connect neighbouring open cells
- Do NOT connect bottom-row cells to vbottom

This prevents bottom-row cells from becoming falsely full through the virtual bottom node.

Conclusion:
The Backwash Bug occurs because the virtual bottom node connects all open bottom-row cells. After percolation, this node is also connected to the virtual top node. Consequently, an open bottom-row cell may incorrectly appear full even without a physical path from the top.

We fix this by using two Union-Find instances: uf-perc contains both virtual nodes and checks percolation, while uf-full contains only the virtual top node and checks fullness. This eliminates backwash.

Time complexity: $O(\alpha(L^2))$ amortized per Union-Find operation.

Q5) The complexity table

Complexity	V1	V2	V3	V4	V5
Union	$O(1)$	$\theta(n)$	$O(n)$	$O(\log n)$	$O(\alpha(n))$
Find / connected	$\theta(n+e)$	$\theta(1)$	$O(n)$	$O(\log n)$	$O(\alpha(n))$
Total time	$\theta(n(n+e))$	$\theta(n^2)$	$\theta(n^2)$	$O(n \log n)$	$O(n\alpha(n))$
Space	$\theta(n+e)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$